



# Práctica 1

- Utilización de la herramienta VIVADO con VHDL
- Diseño y simulación de circuitos sencillos

# Pantalla de bienvenida



Vivado 2019.1

File Flow Tools Window Help Q: Quick Access

**VIVADO**  
HLx Editions

**XILINX**

## Quick Start

Create Project >  
Open Project >  
Open Example Project >

## Tasks

Manage IP >  
Open Hardware Manager >  
Xilinx Tcl Store >

## Learning Center

Documentation and Tutorials >  
Quick Take Videos >  
Release Notes Guide >

### Recent Projects

project\_2  
C:/hlocal/project\_2  
  
test\_VIVADO  
C:/hlocal/test\_VIVADO

Aquí aparecerán los  
proyectos ya  
existentes

Tcl Console

Escribe aquí para buscar



11:53  
06/09/2019



# Crear un proyecto

- Click en “File->Project->New” y después, “Next”.
- Aparecerá la siguiente pantalla

A screenshot of the 'New Project' dialog box. The title bar says 'New Project'. The main section is titled 'Project Name' and contains the instruction 'Enter a name for your project and specify a directory where the project data files will be stored.' There are two input fields: 'Project name:' with the text 'project\_3' and 'Project location:' with the text 'C:/hlocal'. Below these fields is a checked checkbox labeled 'Create project subdirectory'. At the bottom of the main section, it says 'Project will be created at: C:/hlocal/project\_3'. At the bottom of the dialog box are four buttons: a help button with a question mark, '< Back', 'Next >', and 'Cancel'.

Project Name

Enter a name for your project and specify a directory where the project data files will be stored.

Project name: project\_3

Project location: C:/hlocal

☒ Create project subdirectory

Project will be created at: C:/hlocal/project\_3

? < Back Next > Finish Cancel

Siempre en hlocal.  
¡No utilizar espacios u otros  
'caracteres especiales'! (p.ej. ñ, á,  
etc.)

# Crear un proyecto



**New Project**

**Project Type**  
Specify the type of project to create.

☒ **RTL Project**  
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.  
☒ Do not specify sources at this time

☐ **Post-synthesis Project**  
You will be able to add sources, view device resources, run design analysis and implementation.  
☐ Do not specify sources at this time

☐ **I/O Planning Project**  
Do not specify design sources. You will be able to view pin configurations.

☐ **Imported Project**  
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **Example Project**  
Create a new Vivado project from a predefined template.

  **Next >**  

Dejar la opción por defecto "RTL project"



# Seleccionar dónde se va a implementar



New Project

**Default Part**  
Choose a default Xilinx part or board for your project.

Parts **Boards**

[Reset All Filters](#) [Update Board Repositories](#)

Vendor:  Name:  Board Rev:

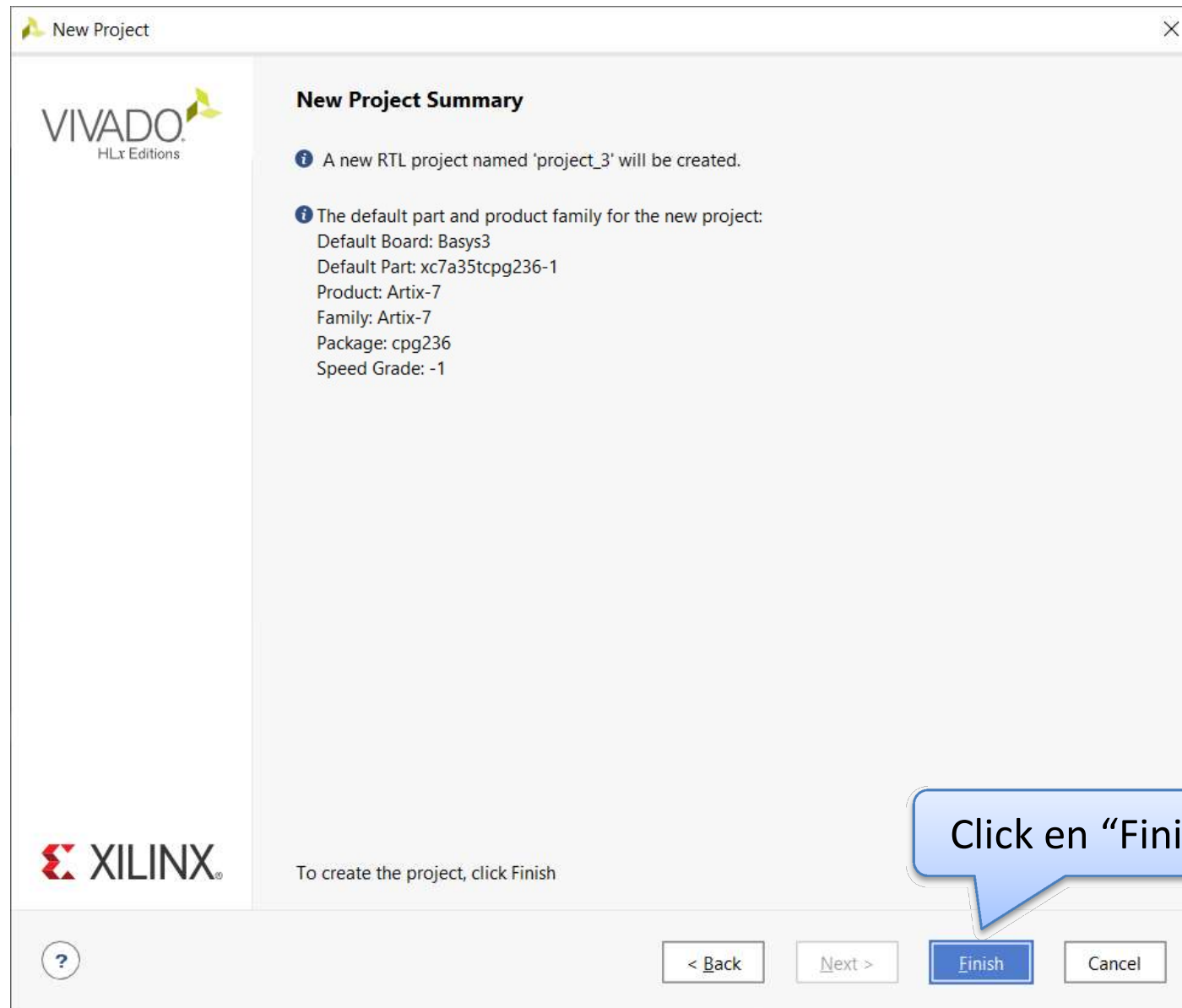
Search:

| Display Name | Preview | Vendor          | File Version | Part            | I/O F |
|--------------|---------|-----------------|--------------|-----------------|-------|
| Basys3       |         | digilentinc.com | 1.1          | xc7a35tcpg236-1 | 236   |

Click en "Boards", y después seleccionar "digilentinc.com" como "Vendor" y "Basys3" como "Name". Click aquí (para seleccionar el dispositivo) y después, click en "Next"

[?](#) [< Back](#) [Next >](#) [Finish](#) [Cancel](#)

# Resumen del proyecto



# Pantalla principal de VIVADO



project\_3 - [C:/hlocal/project\_3/project\_3.xpr] - Vivado 2019.1

File Edit Flow Tools Reports Window Layout View Help Q: Quick Access

Flow Navigator PROJECT MANAGER - project\_3

**PROJECT MANAGER**

- Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- IP INTEGRATOR
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- SIMULATION
  - Run Simulation
- RTL ANALYSIS
  - Open Elaborated Design
- SYNTHESIS
  - Run Synthesis
  - Open Synthesized Design
- IMPLEMENTATION
  - Run Implementation
  - Open Implemented Design
- PROGRAM AND DEBUG

**Sources**

- Design Sources
- Constraints
- Simulation Sources
  - sim\_1
- Utility Sources

Hierarchy Libraries Compile Order

**Properties**

**Project Summary**

Overview | Data

Set

Project name:  
Project location:  
Product family:  
Project part:  
Top module name:  
Target language: Verilog  
Simulator language: Mixed

**Board Part**

Display name: Basys3  
Board part name: digilentinc.com:basys3:part0:1.1  
Board revision: C.0

**Tcl Console**

Name

- synth\_1
- impl\_1

TPWS Total Power Failed Routes LUT FF BRAMs URAM DSP Start Elapsed Run Strategy

Vivado Synthesis Defaults (Vivado Synthesis 2019)  
Vivado Implementation Defaults (Vivado Implementation)

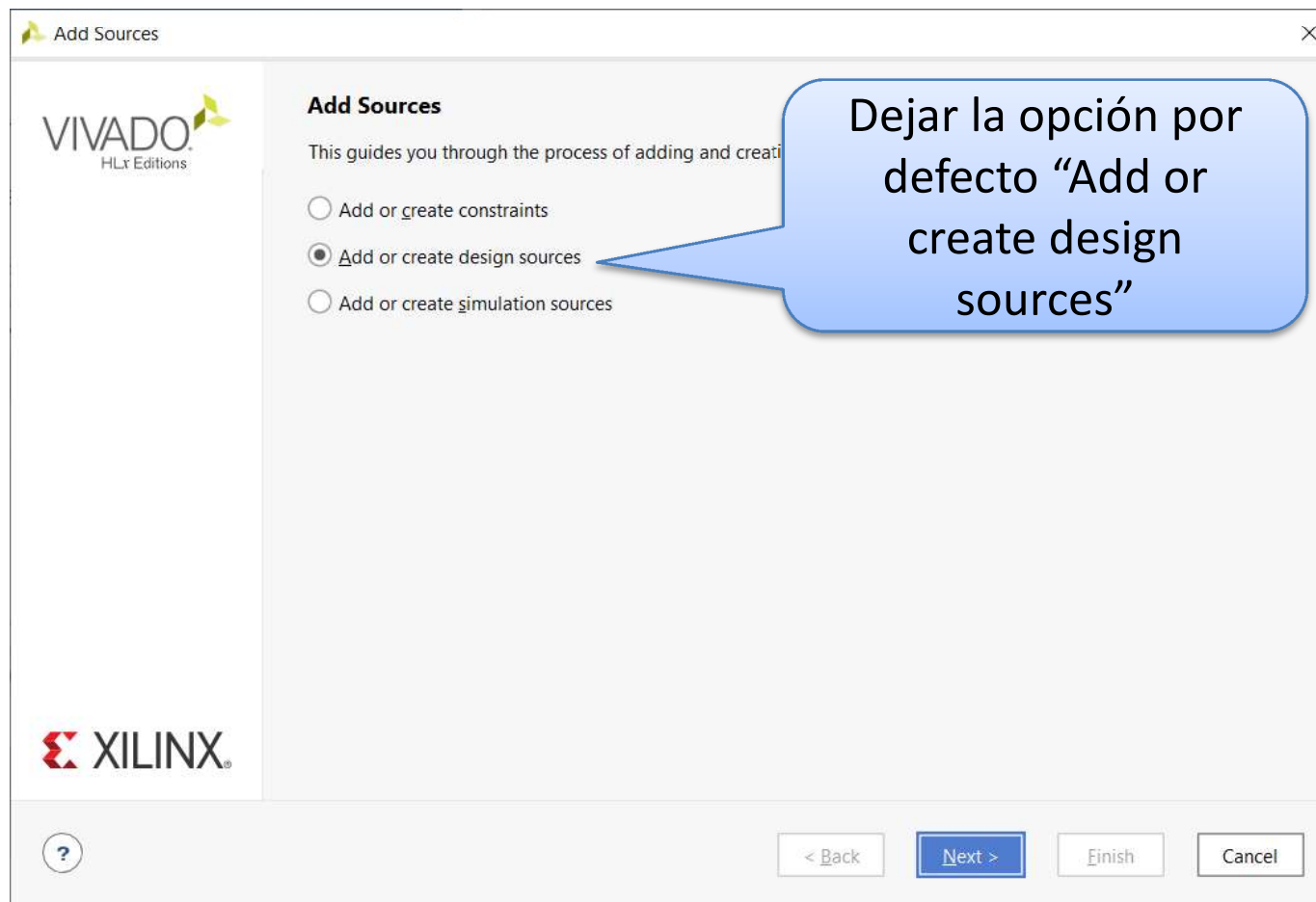
Los ficheros fuente del proyecto están aquí

Las acciones a realizar con el proyecto están aquí

# Añadir o crear nuevos ficheros



- Click en “Add source” (arriba a la izquierda), entonces...



# Añadir o crear nuevos ficheros

**Add Sources**

**Add or Create Design Sources**

Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.

|   | Index | Name        | Library        | Location                                                                              |
|---|-------|-------------|----------------|---------------------------------------------------------------------------------------|
| ● | 1     | adder.vhd   | xil_defaultlib | C:/Users/Juanan/Dropbox/Docencia/2019_2020/TOC-I/2.Laboratory/Lab1/vhdl/1-Intro_files |
| ● | 2     | divider.vhd | xil_defaultlib | C:/Users/Juanan/Dropbox/Docencia/2019_2020/TOC-I/2.Laboratory/Lab1/vhdl/1-Intro_files |

**Buttons:** Add Files, Add Directories, Create File

**Checkboxes:**

- ☐ Scan and add RTL include files into project
- ☒ Copy sources into project
- ☒ Add sources from project directories

**Navigation:** < Back, Next >, Finish, Cancel

Para esta práctica, usar “Add Files”. Después, seleccionar “sumador” y “registro” (disponibles en el Campus Virtual)

Seleccionando “Add Directories”, añadimos todos los ficheros del directorio seleccionado

Seleccionando “Create File”, creamos un nuevo fichero desde cero

**MUY IMPORTANTE:** Mantén la opción “Copy sources into project” ACTIVADA

# Añadir ficheros existentes al proyecto



- Los ficheros añadidos serán visibles en el panel “Sources”:

The screenshot shows the Vivado 2019.1 interface. The 'Sources' panel on the left lists 'adder4b(rtl) (adder.vhd)' and 'clock\_divider(arch\_divider) (divider.vhd)'. The 'Source File Properties' dialog is open for 'adder.vhd', showing the 'Hierarchy' tab. A right-click context menu is visible over the 'adder.vhd' entry, with 'Set as Top' highlighted.

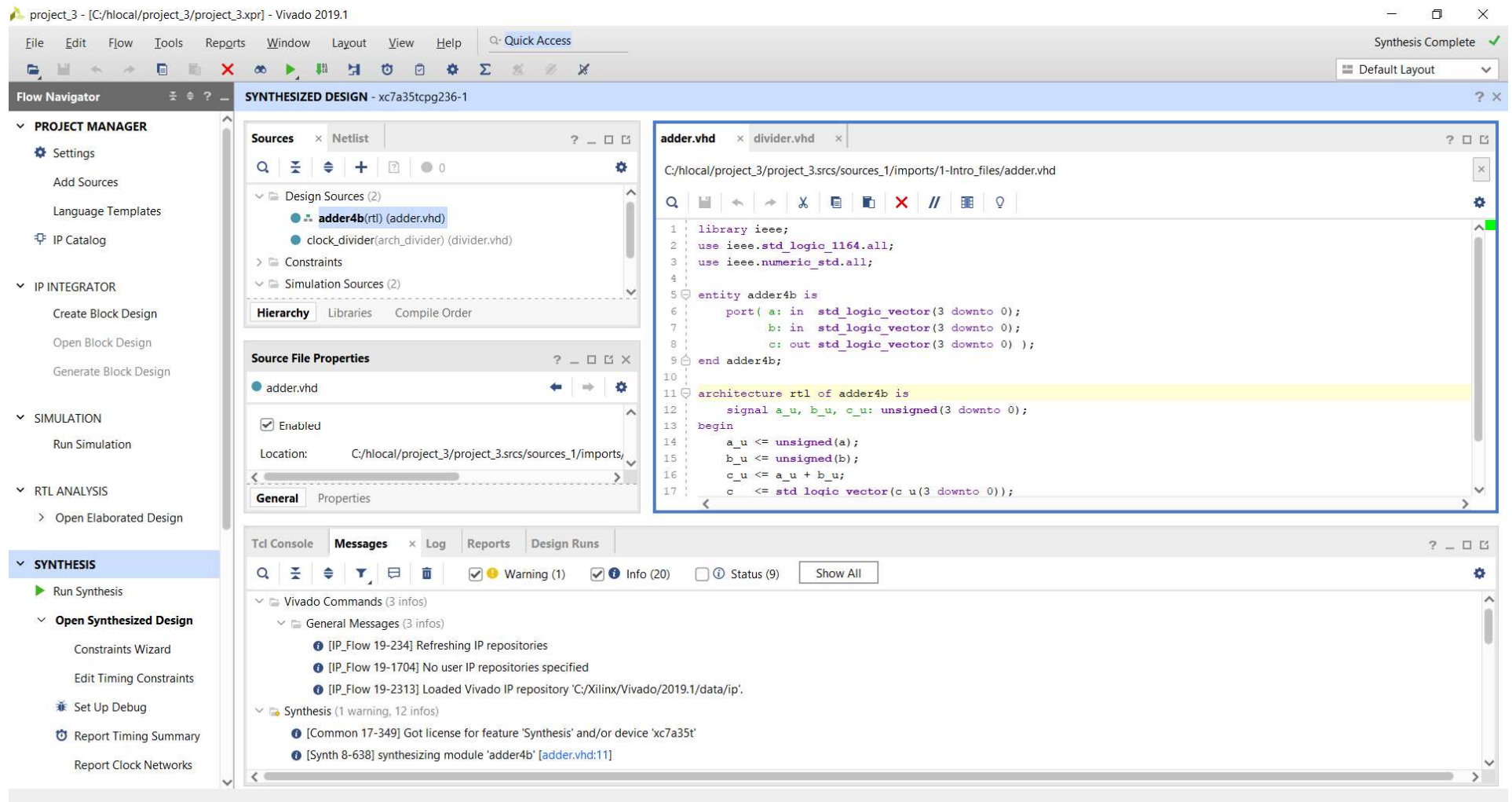
Este símbolo indica que el fichero sumador.vhd es el “top file” en la jerarquía. Este es el archivo sobre el que se realizará la síntesis, implementación, etc.

En cualquier momento, otro fichero puede ser el “top file” haciendo click derecho en él, y después seleccionando “Set as Top”

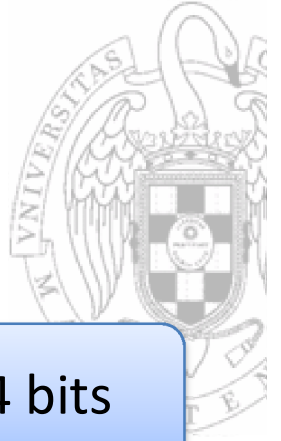


# Añadir ficheros existentes al proyecto

- Haciendo doble click en el fichero, se puede ver y editar el código



# Laboratorio 1.a



```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.std_logic_arith.all;  
use ieee.std_logic_unsigned.all;
```

Este es un sumador de 4 bits

```
entity sumador is  
    port( A: in  std_logic_vector(3 downto 0) ;  
          B: in  std_logic_vector(3 downto 0) ;  
          C: out std_logic_vector(3 downto 0)  
    );  
end sumador;  
  
architecture rtl of sumador is  
  
begin  
    C <= A + B;  
end rtl;
```



# Sintetizando el diseño



project\_3 - [C:/hlocal/project\_3/project\_3.xpr] - Vivado 2019.1

File Edit Flow Tools Reports Window Layout View Help Quick Access

Flow Navigator

- Open Block Design
- Generate Block Design
- ▼ SIMULATION
  - Run Simulation
- ▼ RTL ANALYSIS
  - Open Elaborated Design
- ▼ SYNTHESIS
  - Run Synthesis
  - ▼ Open Synthesized Design
    - Constraints Wizard
    - Edit Timing Constraints
    - Set Up Debug
    - Report Timing Summary
    - Report Clock Networks
    - Report Clock Interaction
    - Report Methodology
    - Report DRC
    - Report Noise
    - Report Utilization
    - Report Power
    - Schematic

SYNTHESIZED DESIGN - xc7a35tcbg236-1

Sources x Netlist

adder.vhd x divider.vhd x

C:/hlocal/project\_3/project\_3.srcs/sources\_1/imports/1-Intro\_files/adder.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder4b is
    port (a: in std_logic_vector(3 downto 0);
          b: in std_logic_vector(3 downto 0);
          c: out std_logic_vector(3 downto 0));
end adder4b;

architecture rtl of adder4b is
    signal a_u, b_u, c_u: unsigned(3 downto 0);
begin
    a_u <= unsigned(a);
    b_u <= unsigned(b);
    c_u <= a_u + b_u;
    c <= std_logic_vector(c_u(3 downto 0));
end;
```

adder.vhd

Enabled

Location: C:/hlocal/project\_3/project\_3.srcs/sources\_1/imports/

General Properties

Tcl Console Messages x Log Reports Design Runs

Warning (1) Info (20) Status (9) Show All

Vivado Commands (3 infos)

- General Messages (3 infos)
- [IP\_Flow 19-234] Refreshing
- [IP\_Flow 19-1704] No user IP
- [IP\_Flow 19-2312]

Synthesis (1 warning)

- [Common 17-349]
- [Synth 8-638] synth

10:0 Insert VHDL

En la pestaña “Flow Navigator”, click en “Run Synthesis”

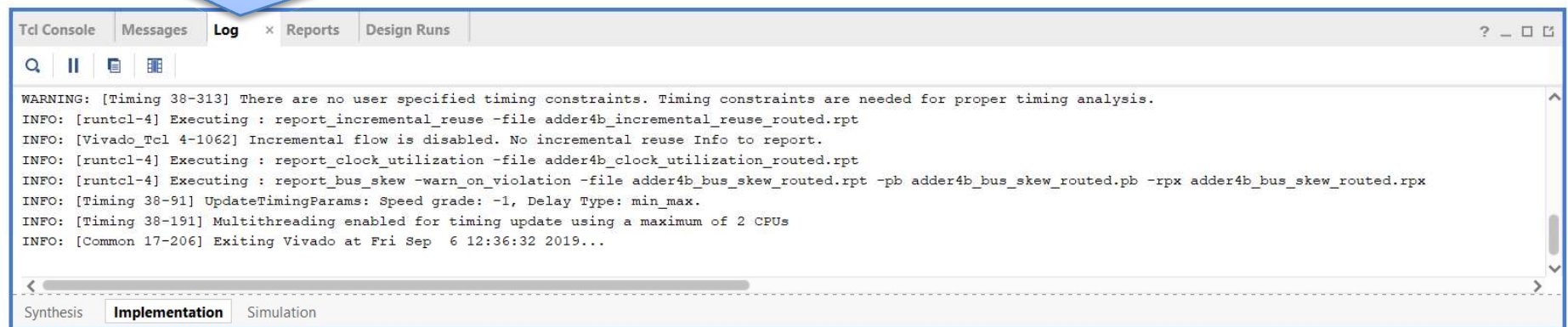
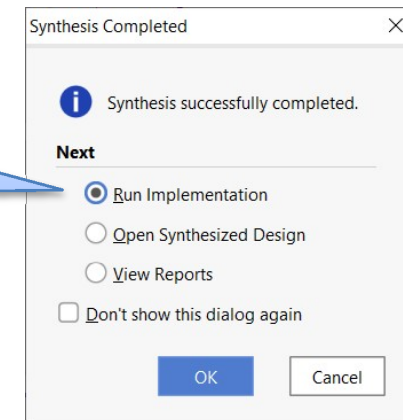
En la pestaña “Log”, se puede ver el progreso del proceso



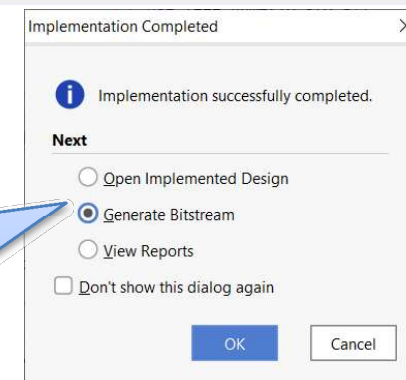
# Sintetizando el diseño

Cuando se ha completado la síntesis, aparecerá la siguiente ventana. Hacer click en “Run Implementation”, después en “OK”.

En la pestaña “Log”, se puede observar que hay mensajes separados para los procesos de síntesis, implementación y simulación.



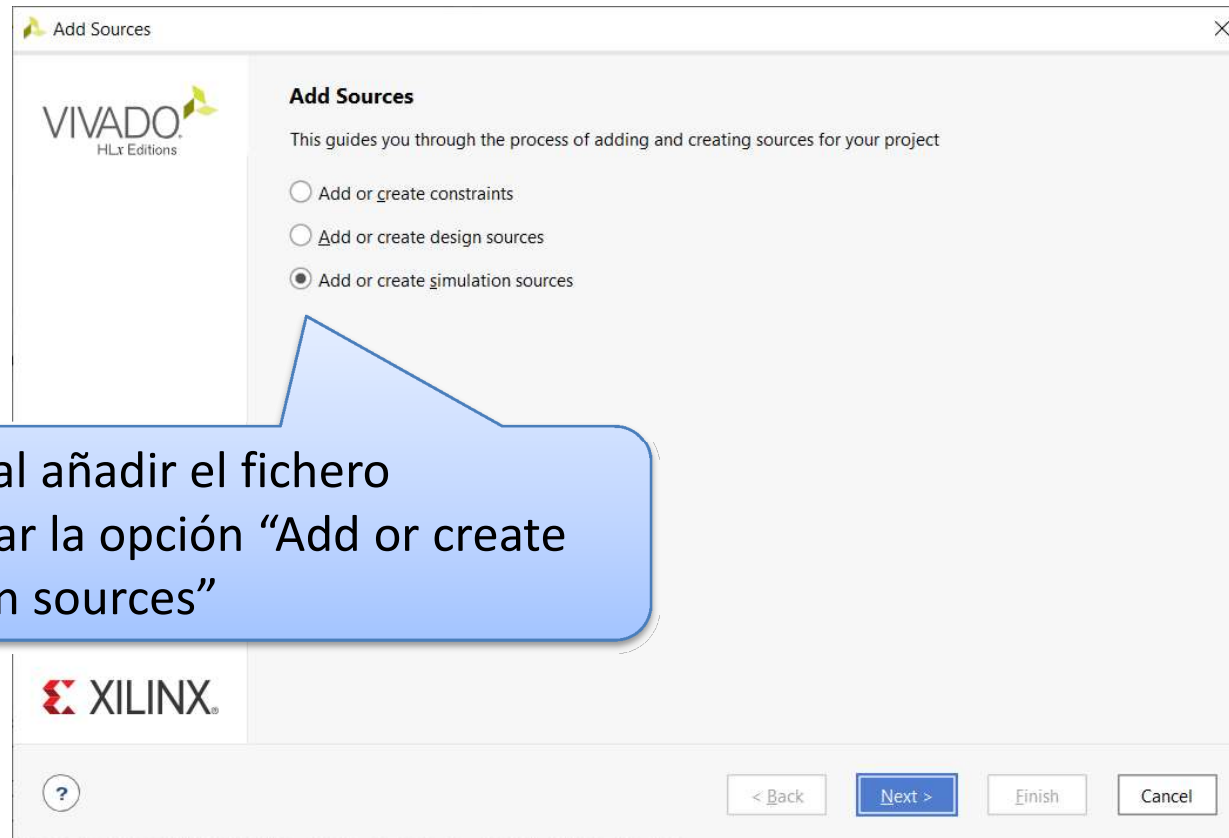
Cuando se ha completado la implementación, aparecerá la siguiente ventana. Si hacemos click en “Generate Bitstream”, DARÁ UN ERROR (es necesario un fichero de restricciones para indicar los pines de E/S).



# Verificar que el diseño es correcto



- Para poder verificar si el diseño es correcto, se debe simular el circuito:
  - Añade el testbench “tb\_sumador.vhd” disponible en el Campus Virtual.



Esta vez, al añadir el fichero seleccionar la opción “Add or create simulation sources”



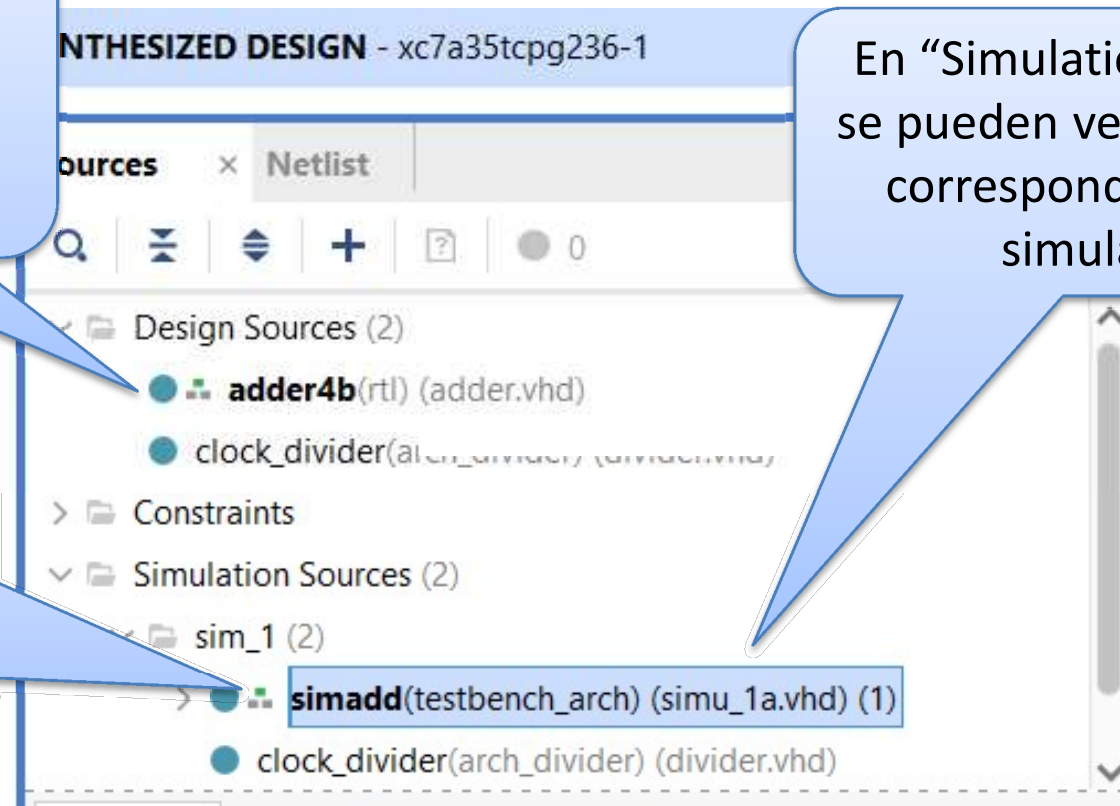
# Simulando el testbench

- Una vez añadido, podemos verlo en el panel “Sources”

En “Design Sources”, se pueden ver los ficheros correspondientes al diseño del circuito

En “Simulation Sources”, se pueden ver los ficheros correspondientes a la simulación

Este símbolo también aparece en uno de los ficheros de simulación. El fichero que tiene el símbolo será el que se simule



# Simulando el testbench



- POSIBLE PROBLEMA: Te has equivocado y has incluido un fichero de simulación como fichero de diseño, o al revés.
  - SOLUCIÓN: En cualquier momento, se puede hacer click sobre el fichero equivocado y seleccionar “Move to Simulation Sources” o “Move to Design Sources” para arreglar este problema.

# Testbench (disponible en el Campus Virtual)



```
-- We add the libraries needed
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Entity declaration
ENTITY tb_sumador IS
END tb_sumador;

-- Architecture
ARCHITECTURE testbench_arch OF tb_sumador IS
    -- Component declaration
    COMPONENT sumador
    PORT (
        A : IN    std_logic_vector(3 downto 0) ;
        B : IN    std_logic_vector(3 downto 0) ;
        C : OUT   std_logic_vector(3 downto 0)
    );
    END COMPONENT;

-- Inputs
signal A : std_logic_vector(3 downto 0) := (others => '0');
signal B : std_logic_vector(3 downto 0) := (others => '0');

-- Outputs
signal C : std_logic_vector(3 downto 0);
```

# Testbench (disponible en el Campus Virtual)



**BEGIN**

-- Instantiation of the unit under test

  ut: sumador **PORT MAP** (

    A => A,

    B => B,

    C => C );

-- Stimuli process

stim\_proc: **process**

**begin**

    A <= "0000";

**wait for** 100 ns;

    A <= "0101";

**wait for** 100 ns;

    A <= "0000";

**wait for** 100 ns;

    A <= "0011";

**wait for** 100 ns;

    A <= "1011";

**wait for** 100 ns;

    A <= "1001";

**wait**;

**end process**;

**END** testbench\_arch;



# Simulación



3- Se puede aumentar el tamaño de esta ventana haciendo click en este símbolo

project\_3 - [C:/hlocal/project\_3/project\_3.xpr] - Vivado 2019.1

File Edit Flow Tools Reports Window Layout View Run Help

Flow Navigator

- PROJECT MANAGER
  - Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- IP INTEGRATOR
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- SIMULATION**
  - Run Simulation
- RTL ANALYSIS
  - Open Elaborated Design
- SYNTHESIS
  - Run Synthesis
  - Open Synthesized Design
    - Constraints Wizard
    - Edit Timing Constraints
    - Set Up Debug
    - Report Timing Summary
    - Report Clock Networks

**SIMULATION - Behavioral Simulation - Functional - sim\_1 - simadd**

Scope Sources

| Name   | Design Unit           | Block T... |
|--------|-----------------------|------------|
| simadd | simadd(testbench_arch | VHDL E     |
| uut    | adder4b(rtl)          | VHDL E     |

Objects Protocol Instanc

| Name   | Value | Data T... |
|--------|-------|-----------|
| A[3:0] | 9     | Array     |
| B[3:0] | 6     | Array     |
| C[3:0] | f     | Array     |

adder.vhd divider.vhd simu\_1a.vhd Untitled 1

| Name   | Value |
|--------|-------|
| A[3:0] | 9     |
| B[3:0] | 6     |
| C[3:0] | f     |

999,996 ps 999,998 ps 1,000,000 ps

9 6 f

Tcl Console

```
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'simadd_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:19 . Memory (MB): peak = 1811.117 ; gain = 4.730
```

Type a Tcl command here

1- En la parte de SIMULATION, click en "Run Simulation", después "Run Behavioral Simulation".

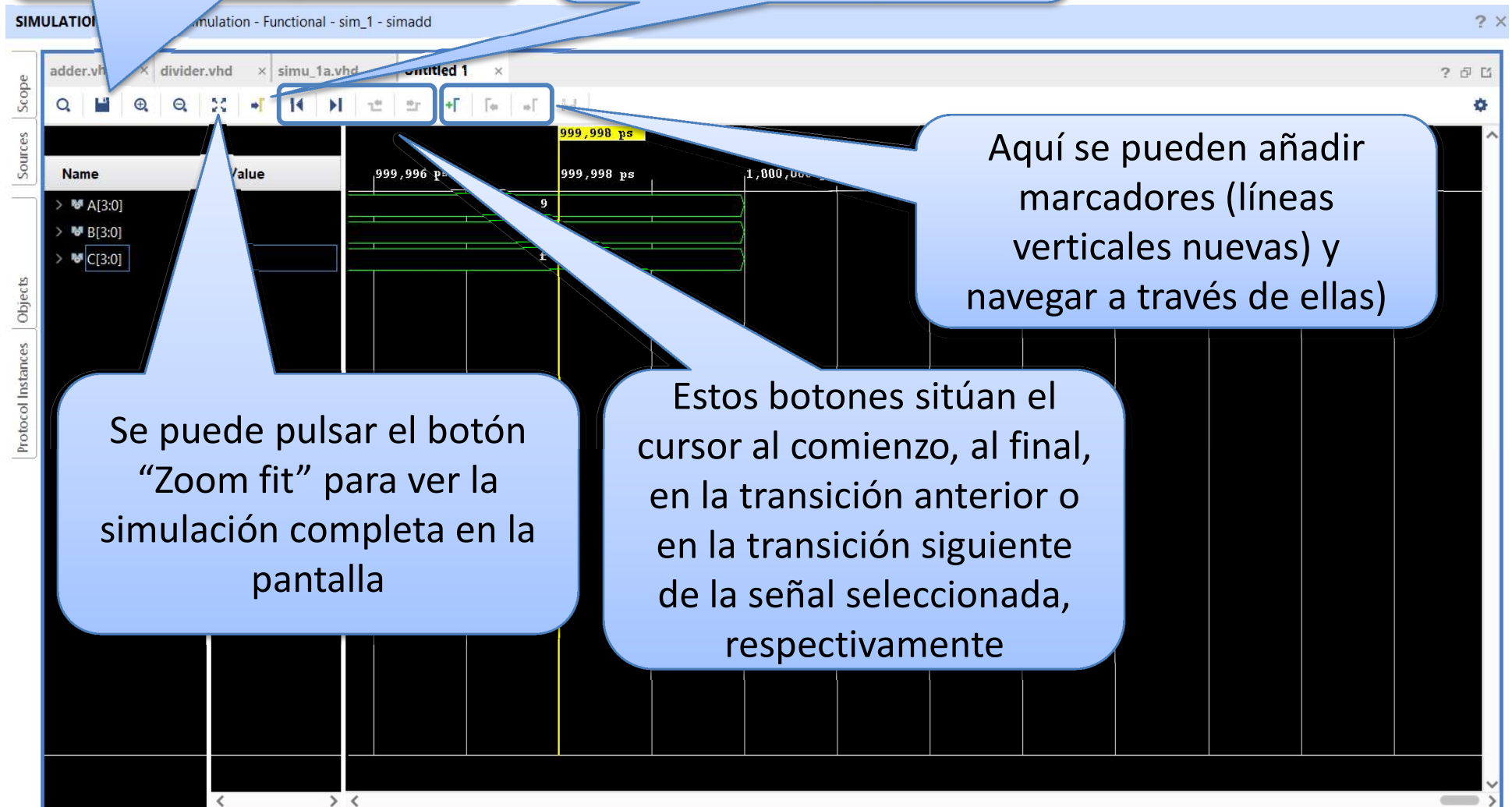
2- Los resultados de la simulación se mostrarán en esta ventana



# Ventana de simulación

Las formas de onda pueden guardarse como un fichero .wcfg

Esto centra la vista en el "cursor" (la línea vertical amarilla)



# Simulación



project\_3 - [C:/hlocal/project\_3/project\_3.xpr] - Vivado 2019.1

File Edit Flow Tools Reports Window Layout View Run Help Quick Access

Flow Navigator

- PROJECT MANAGER
  - Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- IP INTEGRATOR
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- SIMULATION
  - Run Synthesis
  - Open Synthesized Design
    - Constraints Wizard
    - Edit Timing Constraints
    - Set Up Debug
    - Report Timing Summary
    - Report Clock Networks

Scope

Sources

Objects

Instances

sim\_1 - simadd

adder.vhd divider.vhd simu\_1 add simadd\_behav.wcfg\*

Name Value

- A[3:0] 5 0
- B[3:0] 4 7
- C[3:0] 9 7

100 ns 200 ns 900 ns

Se puede hacer click aquí para reiniciar la simulación

Se puede hacer click aquí para simular el testbench completo

Se puede hacer click aquí para simular durante un periodo específico de tiempo (en este caso, 10 us)

Tcl Console Messages Log

Sim Time: 10500 ns

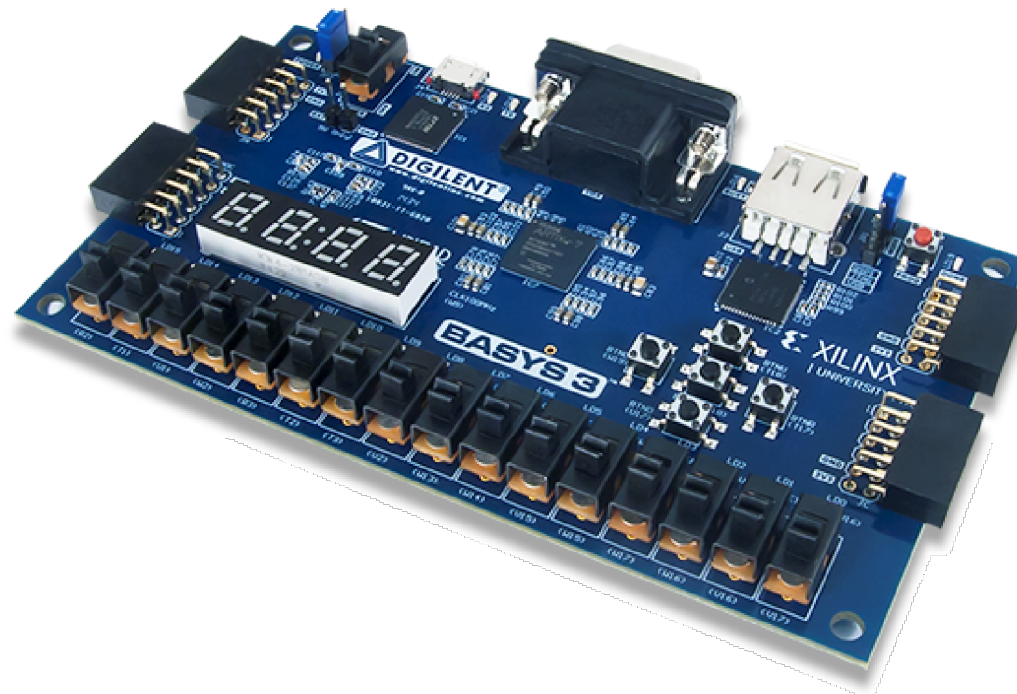


# Implementación

- Se puede comprobar que el circuito funciona en el *mundo real*
  - El circuito se implementará en la FPGA
  - La herramienta de implementación necesita conocer de donde vienen las entradas (pulsadores o switches) y cómo se muestran las salidas (LEDs o displays de 7 segmentos).
    - Debe añadirse un fichero de restricciones (con extensión .xdc) con toda esta información.

# Implementación

- Este es el modelo de placa de FPGA que se utilizará en los laboratorios:



<https://reference.digilentinc.com/reference/programmable-logic/basys-3/start>





# Implementación

- Fichero .xdc
  - Puedes encontrar un fichero completo .xdc para esta placa en el Campus Virtual (descargado de <https://reference.digilentinc.com/reference/programmable-logic/basys-3/start> → Master XDC files).
- Este es un fichero genérico que permite asociar nuestras entradas y salidas con las entradas y salidas físicas de los dispositivos de la placa (LEDs, pulsadores, etc...).
- Todo el código está escrito en los comentarios. Se pueden descomentar las líneas deseadas para usar los dispositivos de E/S que sean necesarios.
  - Cada pin de E/S de la placa está asociado con dos líneas del fichero que empiezan por *set\_property*.
- Por ejemplo, para usar un switch para el pin de entrada A[0] (*sumador.vhd*), sería de la siguiente manera:

```
Project Summary x pins.xdc * x
C:/hlocal/project_3/project_3.srscs/constrs_1/imports/1-Intro_files/pins.xdc

4  ## - rename the used ports (in each line, after get_ports) according to
5
6  ## Clock signal
7  #set_property PACKAGE_PIN W5 [get_ports clk]
8  #set_property IOSTANDARD LVCMOS33 [get_ports clk]
9  #create_clock -add -name sys_clk_pin -period 10.00 -waveform [get_ports clk]
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports {A[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
14 #set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
15 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
16 #set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
```

# Implementación



A y B (2 entradas de 4 bits) se asocian a 8 switches de esta manera

C (una salida de 4 bits) se asocia a 4 LEDs de esta manera

```
## LEDs
set_property PACKAGE_PIN U16 [get_ports {C[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {C[0]}]
set_property PACKAGE_PIN E19 [get_ports {C[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {C[1]}]
set_property PACKAGE_PIN U19 [get_ports {C[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {C[2]}]
set_property PACKAGE_PIN V19 [get_ports {C[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {C[3]}]
```

```
Project Summary x pins.xdc * x
C:/hlocal/project_3/project_3.srsrcs/constrs_1/imports/1-Intro_files/pins.xdc

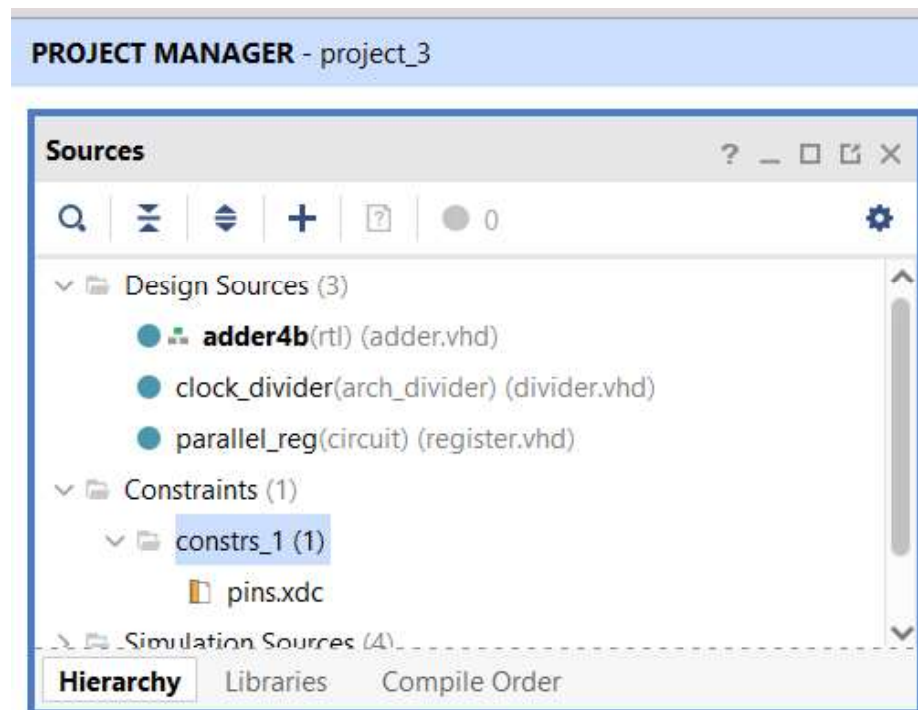
7 #set_property PACKAGE_PIN W5 [get_ports clk]
8 #set_property IOSTANDARD LVCMOS33 [get_ports clk]
9 #create_clock -add -name sys_clk_pin -period 10.00 -wat
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports {A[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
14 set_property PACKAGE_PIN V16 [get_ports {A[1]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
16 set_property PACKAGE_PIN W16 [get_ports {A[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]
18 set_property PACKAGE_PIN W17 [get_ports {A[3]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]
20 set_property PACKAGE_PIN W15 [get_ports {B[0]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
22 set_property PACKAGE_PIN V15 [get_ports {B[1]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
24 set_property PACKAGE_PIN W14 [get_ports {B[2]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
26 set_property PACKAGE_PIN W13 [get_ports {B[3]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]
28 #set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
29 #set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
30 #set_property PACKAGE_PIN T3 [get_ports {sw[9]}]
```





# Implementación

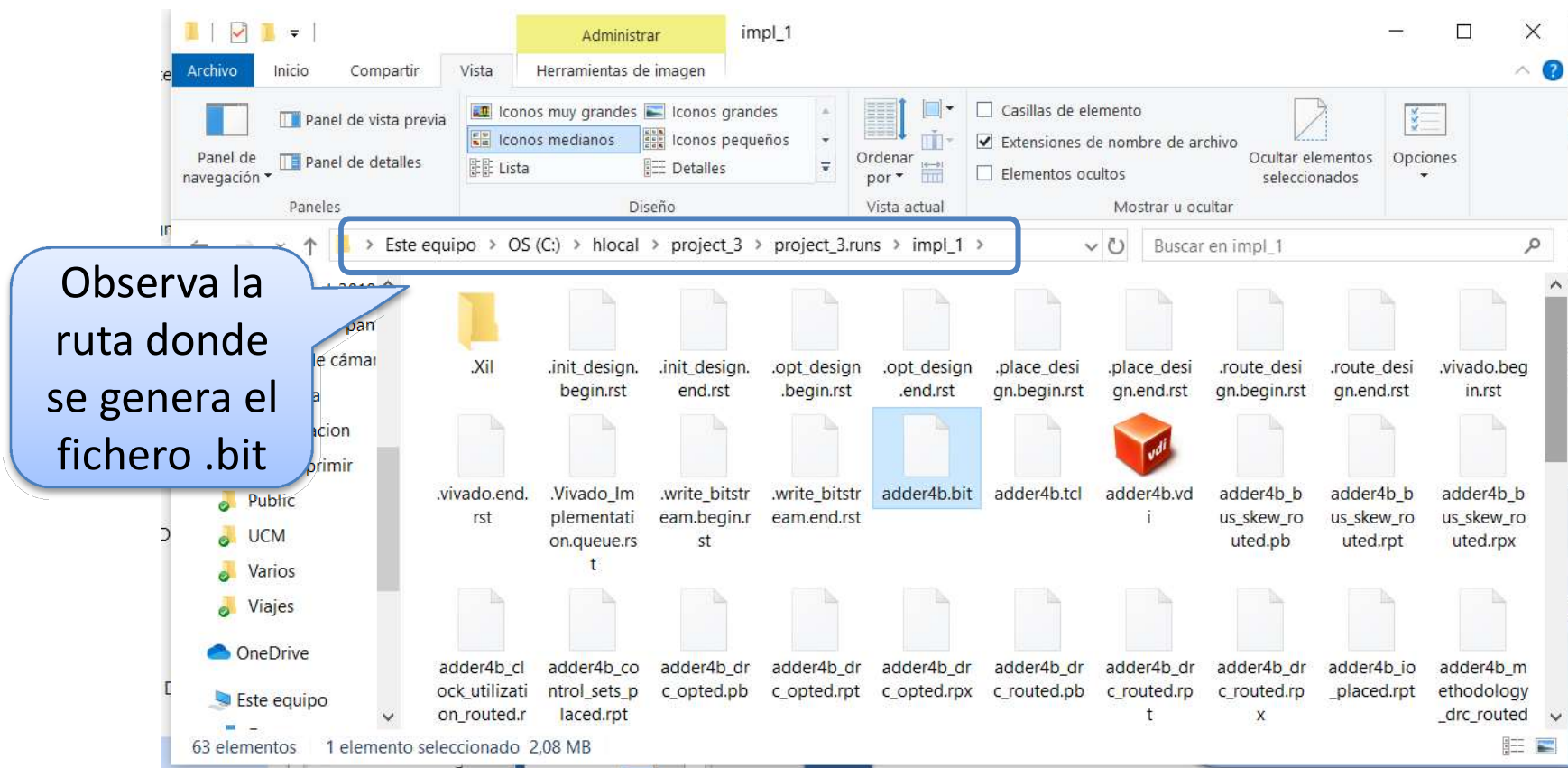
- El fichero *pins.xdc* se añade al proyecto haciendo click en “Add Sources” → “Add or Create Constraints”
- Este fichero será visible en el panel “Sources”.



# Implementación



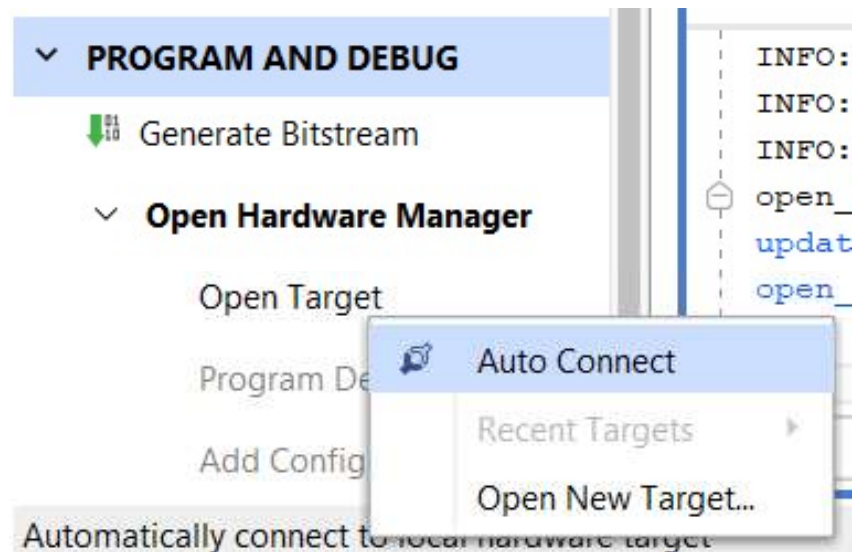
- Los últimos pasos son hacer click en la opción “Run Implementation” (abajo a la izquierda en la pantalla principal) y finalmente, “Generate bitstream”. Cuando acabe, click en “Open Hardware manager”
  - Este último paso genera un fichero de implementación (*top\_entity\_name.bit*), que puede utilizarse para programar la FPGA.



# Descargando el .bit en la FPGA



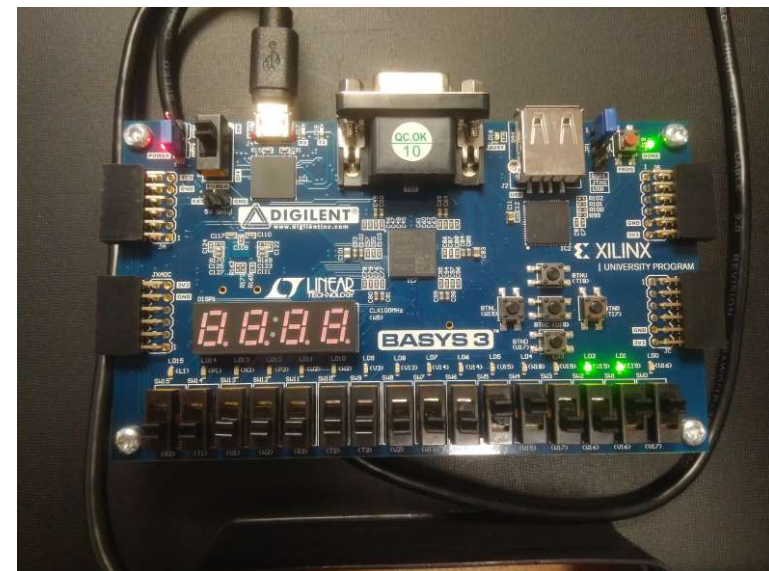
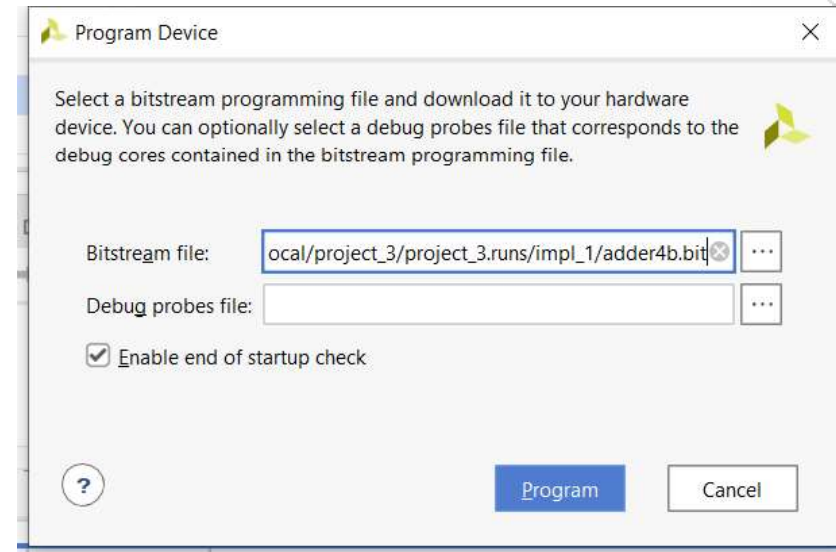
1. Si todavía no lo has hecho, haz click en “Open Hardware Manager”, después click en “Target → Auto Connect”
2. Click en “Program Device → xc7a35t\_0”



# Descargando el .bit en la FPGA



- finalmente, se debe seleccionar un fichero .bit (el fichero .bit del proyecto abierto se selecciona por defecto). Click en “Program”.
- El bitstream se descargará en la FPGA y se podrá comprobar si el funcionamiento es el esperado (prueba a mover SW7-SW0 para comprobar si el resultado de la suma es visible en LD3-LD0).
- Para volver a editar un fichero, click en “PROJECT MANAGER” (arriba a la izquierda de la ventana principal).



# Laboratorio 1.b



- Simular e implementar un registro de 4 bits con entrada paralela / salida paralela.

# Laboratorio 1.b



-----  
-- register with parallel input / parallel output  
-----

```
library IEEE;
```

```
use IEEE.std_logic_1164.all;
```

```
entity registro is
```

```
    port(rst    : in  std_logic;
```

```
          clk   : in  std_logic;
```

```
          load  : in  std_logic;
```

```
          E     : in  std_logic_vector(3 downto 0);
```

```
          S     : out std_logic_vector(3 downto 0)
```

```
    );
```

```
end registro;
```

Fichero registro.vhd



# Laboratorio 1.b



architecture Behavioral of registro is  
begin

```
    process(clk)
    begin
        if (rising_edge(clk)) then
            if rst = '1' then
                S <= "0000";
            elsif load = '1' then
                S <= E;
            end if;
        end if;
    end process;
end Behavioral;
```

Fichero registro.vhd

# Implementación Laboratorio 1.b



- Pin del reloj:

Descomentar estas 3 líneas (borrando las #’s)

Asegurarse que vuestra señal de reloj se llama exactamente como aparece aquí (en este caso, clk)

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

Esta línea indica que clk es una señal de reloj de periodo 10.00 ns (el oscilador en el pin W5 genera dicha señal de reloj).

- Además, asegurarse de que tenéis 4 switches (para la entrada), 2 pulsadores (1 para reset y otro para load) y 4 LEDs (para la salida).